

UNISENAI
Tecnologia em Análise e Desenvolvimento de Sistemas

Arthur Moreira
Douglas do Carmo
Jamil Cherem

SMARTRENT B2B
Plataforma de precificação preditiva de diárias para pequenos anfitriões e administradores
de imóveis da Grande Florianópolis

Florianópolis
2026

Arthur Moreira
Douglas do Carmo
Jamil Cherem

SMARTRENT B2B

Plataforma de precificação preditiva de diárias para pequenos anfitriões e administradores de imóveis da Grande Florianópolis

Relatório técnico apresentado à disciplina de Projeto Aplicado IV, do curso de Tecnologia em Análise e Desenvolvimento de Sistemas do UniSENAI, como parte da avaliação da unidade curricular.

Orientadora: Profa. Milena Maredmi Correa

Florianópolis
2026

RESUMO

Pequenos anfitriões e administradores de imóveis de temporada da Grande Florianópolis ainda controlam reservas por planilha ou aplicativo de mensagens e definem o preço da diária por intuição, sem levar em conta a sazonalidade ou a localização específica do imóvel. Esse cenário favorece a ocorrência de reservas sobrepostas para o mesmo período e mantém os imóveis ociosos fora da alta temporada. Este trabalho descreve o desenvolvimento do SmartRent B2B, uma plataforma para gestão de imóveis e reservas que inclui uma verificação automática de conflito de datas e um mecanismo de sugestão de preço baseado em inteligência artificial generativa, com um cálculo alternativo determinístico para os casos em que o serviço de IA não responde. O sistema foi construído em Java 17 e Spring Boot, com persistência em PostgreSQL, e alinha-se ao Objetivo de Desenvolvimento Sustentável 8 da Organização das Nações Unidas ao oferecer a um microempreendedor uma ferramenta de gestão antes restrita a operações de maior porte. São apresentados o levantamento de requisitos, a modelagem do domínio, os diagramas UML da solução e o estado atual da implementação.

Palavras-chave: SmartRent B2B; precificação dinâmica; gestão de reservas; sistemas distribuídos; Objetivo de Desenvolvimento Sustentável 8.

SUMÁRIO

1 INTRODUÇÃO	5
1.1 Contextualização e problema	5
1.2 Solução proposta	5
1.3 Alinhamento com o Objetivo de Desenvolvimento Sustentável 8	5
2 PERSONAS	7
2.1 Marina Andrade, pequena anfitriã	7
2.2 Carlos Bittencourt, administrador de imóveis	7
3 REQUISITOS	8
3.1 Requisitos funcionais	8
3.2 Requisitos não funcionais	9
4 CASOS DE USO	10
4.1 Gerenciar reservas e verificar conflito de datas	10
4.2 Gerar sugestão de preço com mecanismo de contingência	10
4.3 Demais casos de uso	11
5 DIAGRAMAS UML	12
5.1 Diagrama de classes	12
5.2 Diagrama de sequência da criação de reserva	12
5.3 Diagrama de sequência da sugestão de preço	13
6 MODELO DE DADOS	14
6.1 Modelo lógico	14
6.2 Modelo físico	14
7 TELAS	17
REFERÊNCIAS	18

1 INTRODUÇÃO

1.1 Contextualização e problema

A maior parte dos pequenos anfitriões e administradores de imóveis de temporada da Grande Florianópolis ainda depende de processos manuais para operar seus negócios: reservas anotadas em planilha ou combinadas por aplicativo de mensagens, e preço de diária definido por intuição ou por comparação informal com anúncios semelhantes. Não há, nesse processo, nenhuma ferramenta que relacione sazonalidade, localização específica do imóvel e perfil da propriedade para indicar um preço competitivo.

Esse modo de operação traz três consequências recorrentes. A primeira é o *overbooking*: sem uma verificação automática, é possível aceitar duas reservas para o mesmo imóvel em datas que se sobrepõem. A segunda é a ociosidade dos imóveis fora da alta temporada, decorrente da ausência de uma estratégia de preço que reaja à demanda ao longo do ano. A terceira é a exclusão do pequeno empreendedor das ferramentas de revenue management hoje disponíveis no mercado, que costumam ser corporativas, caras e voltadas a redes hoteleiras de grande porte, não a quem administra um punhado de imóveis.

1.2 Solução proposta

O SmartRent B2B é uma plataforma para gestão de imóveis e reservas de temporada, desenvolvida com escopo delimitado por uma matriz MoSCoW para caber em um ciclo de 30 dias. O sistema mantém um cadastro de imóveis e reservas com verificação automática de conflito de datas, calcula uma sugestão de preço de diária combinando sazonalidade, microgeografia (latitude e longitude do imóvel) e perfil da propriedade por meio de inteligência artificial generativa, e recorre a uma regra de negócio determinística sempre que essa IA está indisponível, registrando qual dos dois caminhos gerou cada sugestão. Um *dashboard* reúne indicadores de ocupação e de evolução de preço para o anfitrião. A autenticação de usuários é reaproveitada do Projeto Aplicado III, evitando retrabalho em um domínio já resolvido pela equipe em semestre anterior.

1.3 Alinhamento com o Objetivo de Desenvolvimento Sustentável 8

O projeto se relaciona com o Objetivo de Desenvolvimento Sustentável 8 da Agenda 2030, voltado ao trabalho decente e ao crescimento econômico (ORGANIZAÇÃO DAS NAÇÕES UNIDAS, 2015), em três de suas metas.

Meta	Contribuição do SmartRent B2B
8.2 — diversificação, modernização tecnológica e inovação produtiva	Leva a um microempreendedor de temporada um instrumento de precificação orientado a dados, hoje restrito a redes hoteleiras de maior porte.
8.3 — apoio às micro e pequenas empresas e ao empreendedorismo	Substitui o controle informal por planilha e mensagens por um registro único de reservas e indicadores, aproximando a operação do anfitrião de uma gestão profissional.
8.9 — turismo sustentável, geração de emprego e valorização da cultura local	A sugestão de preço sensível à sazonalidade favorece uma distribuição mais equilibrada da ocupação ao longo do ano, reduzindo a ociosidade típica da baixa temporada no litoral catarinense.

Fonte: elaborado pelos autores (2026).

2 PERSONAS

As personas a seguir sintetizam os dois perfis de usuário considerados no levantamento de requisitos do MVP.

2.1 Marina Andrade, pequena anfitriã

Marina tem 34 anos, é professora de yoga e administra três imóveis de temporada próprios, em Canasvieiras e em Jurerê. Ela controla as reservas por uma planilha compartilhada e por conversas de WhatsApp com os hóspedes, e já enfrentou um caso de *overbooking*: aceitou duas reservas para o mesmo apartamento no mesmo fim de semana. O preço da diária é definido observando anúncios parecidos em outras plataformas, sem um critério mais sistemático. Marina não tem tempo nem conhecimento técnico para operar uma ferramenta de revenue management corporativa; do SmartRent B2B ela espera sobretudo a garantia de que duas reservas nunca vão colidir e uma sugestão de preço simples, que possa aceitar ou ajustar sem precisar entender a lógica estatística por trás do número.

2.2 Carlos Bittencourt, administrador de imóveis

Carlos tem 45 anos, é ex-corretor de imóveis e administra uma carteira de catorze imóveis de terceiros na Grande Florianópolis, repassando parte da receita aos proprietários. Hoje mantém três planilhas separadas, para reservas, repasses e manutenção, que com frequência saem de sincronia entre si. Sua precificação é reativa: só reduz o valor da diária quando percebe queda na procura, geralmente tarde demais para recuperar a ocupação do período. Carlos já avaliou contratar um PMS (*property management system*) internacional, mas considerou as opções do mercado caras e complexas para o porte da sua operação. Do sistema, ele espera um painel único com visão consolidada de todos os imóveis e uma justificativa auditável para cada preço sugerido, que possa apresentar aos proprietários.

Uma persona secundária, beneficiada de forma indireta, é o próprio hóspede: ele se beneficia de um preço mais coerente com a demanda e de menor risco de encontrar sua reserva cancelada por conflito de agenda, ainda que não interaja diretamente com o sistema no escopo deste MVP.

3 REQUISITOS

Os requisitos foram organizados segundo a matriz MoSCoW (Must, Should, Could, Won't) definida para o MVP de 30 dias; itens classificados como Won't foram registrados como Visão Futura e não fazem parte do escopo atual.

3.1 Requisitos funcionais

Quadro 2 – Requisitos funcionais do MVP

ID	Descrição	MoSCoW
RF01	O sistema deve permitir que o anfitrião se autentique com as credenciais e a infraestrutura de autenticação herdadas do Projeto Aplicado III.	Must
RF02	O sistema deve permitir cadastrar um imóvel com título, tipo, endereço completo (incluindo latitude e longitude), capacidade, número de quartos e banheiros, valor de diária base e comodidades.	Must
RF03	O sistema deve permitir editar os dados de um imóvel e inativá-lo.	Must
RF04	O sistema deve permitir listar os imóveis de um anfitrião, com filtro por situação ativa ou inativa.	Must
RF05	O sistema deve permitir cadastrar uma reserva vinculada a um imóvel, com os dados do hóspede e o período de entrada e saída.	Must
RF06	O sistema deve impedir o cadastro ou a edição de uma reserva cujo período se sobreponha a outra reserva ativa do mesmo imóvel.	Must
RF07	O sistema deve permitir cancelar ou concluir uma reserva, atualizando sua situação.	Must
RF08	O sistema deve calcular uma sugestão de preço de diária para um imóvel em uma data de referência, considerando sazonalidade e microgeografia, por meio de inteligência artificial generativa.	Must
RF09	O sistema deve aplicar uma regra de negócio determinística quando a IA generativa estiver indisponível ou o tempo de resposta for excedido, registrando a origem de cada cálculo.	Must
RF10	O sistema deve exibir um painel com indicadores de ocupação e evolução dos preços sugeridos por imóvel.	Should
RF11	O sistema deve permitir consultar o histórico de sugestões de preço de um imóvel em um intervalo de datas.	Should
RF12	Ao recusar uma reserva por conflito de datas, o sistema deve indicar quais reservas existentes causaram o conflito.	Should
RF13	Sincronização bidirecional de reservas com plataformas externas via iCal.	Won't (Visão Futura)
RF14	Processamento de pagamentos reais por meio de um gateway.	Won't (Visão Futura)

Fonte: elaborado pelos autores (2026).

3.2 Requisitos não funcionais

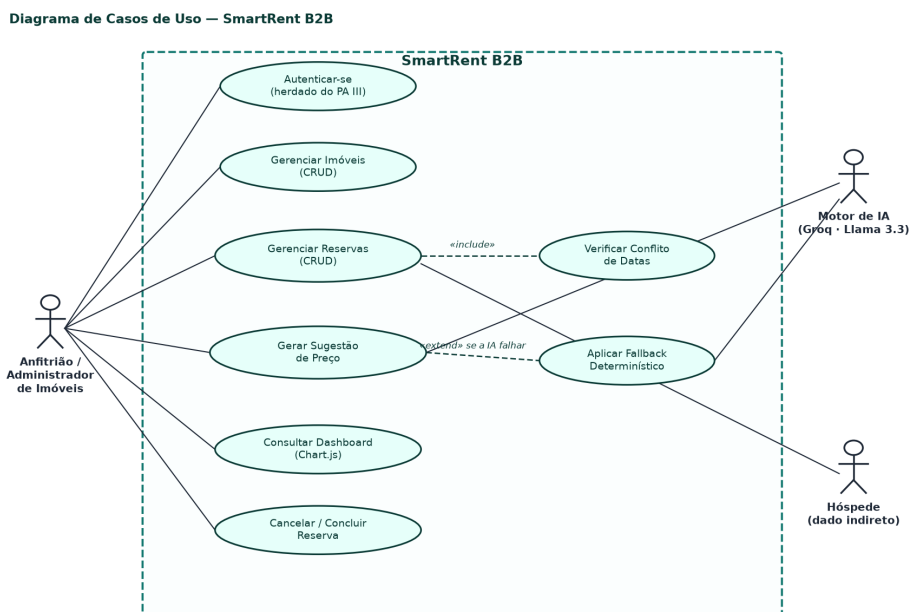
Quadro 3 – Requisitos não funcionais do MVP

ID	Descrição	Categoria
RNF01	O backend deve ser implementado em Java 17 com Spring Boot 3, usando Spring Web, Spring Data JPA e o cliente HTTP nativo (RestClient).	Tecnologia
RNF02	Os dados devem ser persistidos em PostgreSQL, gerenciado por meio do Supabase.	Tecnologia
RNF03	Toda credencial ou chave de API deve ser fornecida ao sistema por variável de ambiente, nunca escrita diretamente no código-fonte.	Segurança
RNF04	As chamadas à IA generativa e a quaisquer serviços externos devem ser simuladas (mock) nos testes automatizados, de modo que o build não dependa de rede.	Testabilidade
RNF05	O esquema do banco de dados deve ser versionado por migration; em produção, o Hibernate deve apenas validar o esquema existente, nunca alterá-lo automaticamente.	Manutenibilidade
RNF06	A interface deve ser responsiva, construída com HTML5 semântico, JavaScript assíncrono e Tailwind CSS, com prioridade ao layout para dispositivos móveis.	Usabilidade
RNF07	As operações de cadastro, consulta, alteração e remoção devem responder em tempo compatível com uso interativo, excluía a latência da chamada externa à IA.	Desempenho
RNF08	O código deve seguir os princípios SOLID e a separação em camadas de controlador, serviço e repositório, com objetos de transferência de dados implementados como Records.	Manutenibilidade
RNF09	A esteira de integração contínua deve executar a compilação e os testes a cada envio de código, impedindo a integração de alterações que quebrem o build.	Confiabilidade

Fonte: elaborado pelos autores (2026).

4 CASOS DE USO

Figura 1 – diagrama de casos de uso do SmartRent B2B



Fonte: elaborado pelos autores (2026).

4.1 Gerenciar reservas e verificar conflito de datas

Quadro 4 – Caso de uso — gerenciar reservas e verificar conflito de datas

Ator principal	Anfitrião ou administrador de imóveis
Pré-condição	Anfitrião autenticado; imóvel já cadastrado e ativo
Fluxo principal	O anfitrião informa os dados da reserva (imóvel, hóspede, datas de entrada e saída, valor). O sistema verifica se há sobreposição de datas para o mesmo imóvel, desconsiderando reservas já canceladas. Não havendo conflito, a reserva é registrada com situação pendente e o sistema confirma a operação.
Fluxo alternativo	Havendo conflito, o sistema recusa a operação e retorna as reservas que causaram a sobreposição, sem registrar nada.
Pós-condição	A reserva passa a constar na listagem do imóvel e nenhuma sobreposição de datas ativa existe para ele.

Fonte: elaborado pelos autores (2026).

4.2 Gerar sugestão de preço com mecanismo de contingência

Quadro 5 – Caso de uso — gerar sugestão de preço com mecanismo de contingência

Ator principal	Anfitrião ou administrador de imóveis
Ator secundário	Serviço de IA generativa (Groq / Llama 3.3)
Pré-condição	Imóvel cadastrado com endereço geocodificado
Fluxo principal	O anfitrião solicita a sugestão de preço para um imóvel e uma data de referência. O sistema monta o contexto de sazonalidade e microgeografia e consulta a IA generativa, que devolve o valor sugerido, os fatores de ajuste considerados e uma justificativa textual. A sugestão é registrada com a origem marcada como IA generativa.
Fluxo alternativo	Se a chamada à IA falhar ou exceder o tempo limite, o sistema aplica uma regra determinística de contingência e registra a sugestão com a origem marcada como cálculo de contingência, de modo que o anfitrião nunca fique sem um valor sugerido.
Pós-condição	A sugestão fica disponível para consulta e exibição no painel, com a origem do cálculo identificada.

Fonte: elaborado pelos autores (2026).

4.3 Demais casos de uso

Quadro 6 – Demais casos de uso do MVP

Caso de uso	Resumo
Autenticar-se	Login com e-mail e senha, reaproveitando a infraestrutura de segurança do Projeto Aplicado III.
Gerenciar imóveis	Cadastro, edição, listagem e inativação de imóveis.
Consultar painel	Visualização de indicadores de ocupação e evolução de preço sugerido.
Cancelar ou concluir reserva	Atualização da situação de uma reserva; reservas canceladas liberam o período para uma nova reserva.

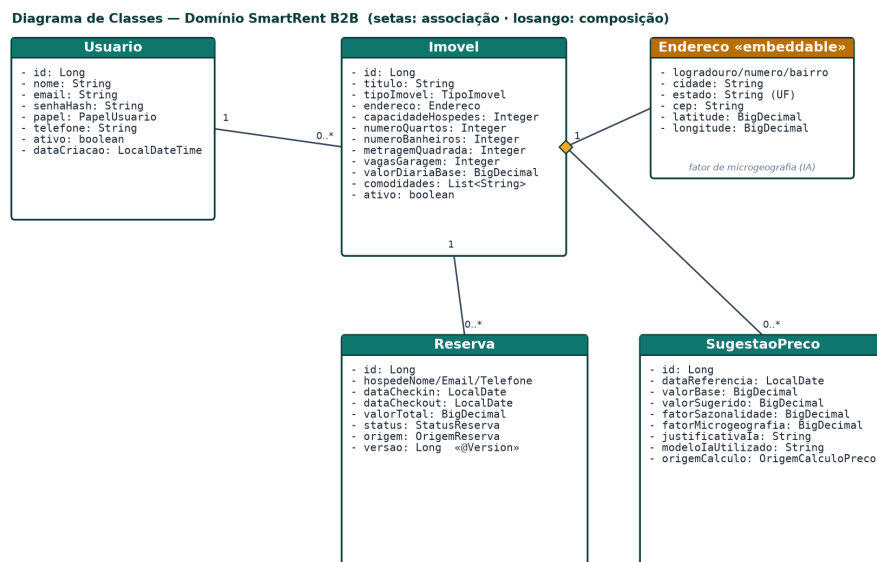
Fonte: elaborado pelos autores (2026).

5 DIAGRAMAS UML

5.1 Diagrama de classes

O diagrama a seguir representa as entidades do domínio e seus relacionamentos: um usuário possui vários imóveis; cada imóvel possui várias reservas e várias sugestões de preço ao longo do tempo. O endereço é modelado como um objeto de valor incorporado ao imóvel, pois é nele que residem a latitude e a longitude usadas pelo cálculo de microgeografia.

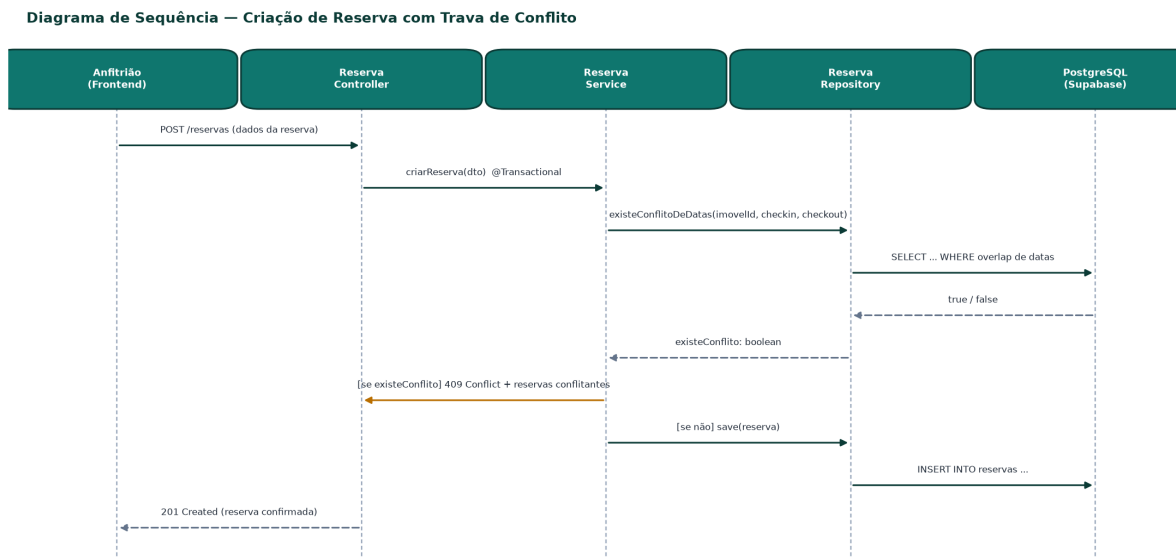
Figura 2 – diagrama de classes do domínio



Fonte: elaborado pelos autores (2026).

5.2 Diagrama de sequência da criação de reserva

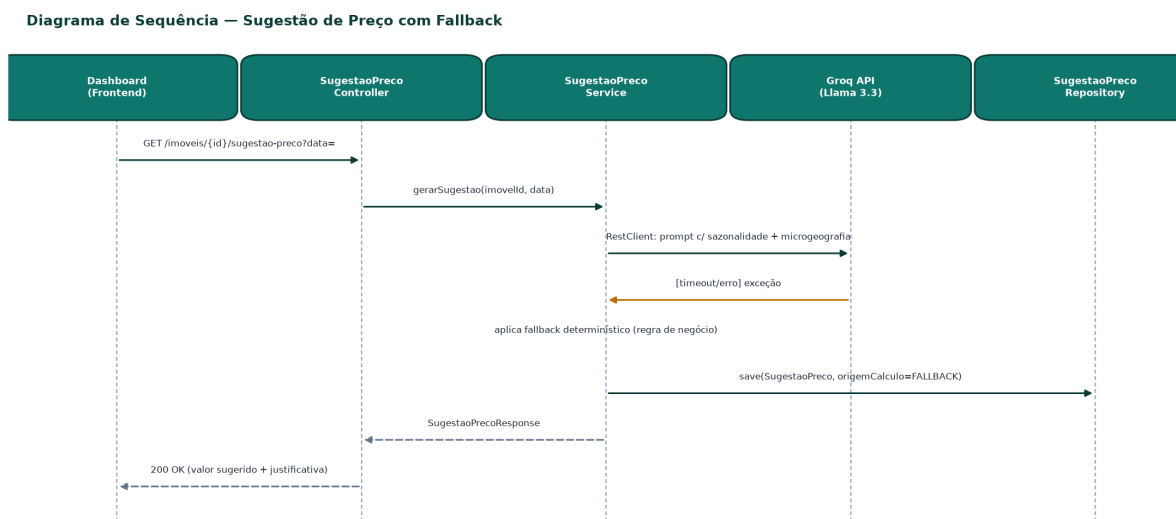
Figura 3 – criação de reserva com verificação de conflito de datas



Fonte: elaborado pelos autores (2026).

5.3 Diagrama de sequência da sugestão de preço

Figura 4 – geração da sugestão de preço com desvio para o cálculo de contingência



Fonte: elaborado pelos autores (2026).

6 MODELO DE DADOS

6.1 Modelo lógico

O modelo lógico corresponde ao diagrama de classes apresentado na Seção 5.1: quatro entidades principais (usuário, imóvel, reserva e sugestão de preço) e um objeto de valor incorporado (endereço). Todos os relacionamentos entre as entidades principais são de um para muitos.

6.2 Modelo físico

O esquema físico, em PostgreSQL, está especificado nos quadros a seguir e encontra-se implementado. Ele foi gerado a partir do mapeamento objeto-relacional em modo somente-geração-de-script do Hibernate (FIELDING; BAUER; KING, [2024] apud documentação do projeto) e é versionado por migrations aplicadas pelo Flyway na inicialização da aplicação, cabendo ao Hibernate apenas validar o esquema existente, nunca alterá-lo. A conformidade entre o mapeamento e os quadros a seguir foi verificada contra uma instância PostgreSQL 16 em ambiente de desenvolvimento.

Quadro 7 – Estrutura da tabela usuarios

Coluna	Tipo	Observação
id	bigserial	chave primária
nome	varchar(120)	obrigatório
email	varchar(150)	obrigatório, único
senha_hash	varchar(60)	obrigatório
papel	varchar(20)	obrigatório
telefone	varchar(20)	opcional
ativo	boolean	obrigatório
data_criacao	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Quadro 8 – Estrutura da tabela imoveis

Coluna	Tipo	Observação
id	bigserial	chave primária
usuario_id	bigint	obrigatório, referencia usuarios
titulo	varchar(150)	obrigatório
descricao	text	opcional
tipo_imovel	varchar(20)	obrigatório

Coluna	Tipo	Observação
endereço (logradouro, bairro, cidade, estado, cep)	varchar	obrigatório
latitude, longitude	numeric(10,7)	obrigatório — microgeografia
capacidade_hospedes, numero_quartos, numero_banheiros	integer	obrigatório
metragem_quadrada	integer	opcional — área útil em m²
vagas_garagem	integer	opcional — nulo e zero são distintos
valor_diaria_base	numeric(10,2)	obrigatório
ativo	boolean	obrigatório
data_cadastro	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Quadro 9 – Estrutura da tabela reservas

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório, referencia imoveis
hospede_nome, hospede_email	varchar	obrigatório
hospede_telefone	varchar(20)	opcional
data_checkin, data_checkout	date	obrigatório
valor_total	numeric(10,2)	obrigatório
status, origem	varchar(20)	obrigatório
observacoes	text	opcional
data_criacao	timestamp	obrigatório
versao	bigint	controle de concorrência otimista

Fonte: elaborado pelos autores (2026).

Quadro 10 – Estrutura da tabela sugestoes_preco

Coluna	Tipo	Observação
id	bigserial	chave primária
imovel_id	bigint	obrigatório, referencia imoveis
data_referencia	date	obrigatório
valor_base, valor_sugerido	numeric(10,2)	obrigatório
percentual_ajuste, fator_sazonalidade, fator_microgeografia	numeric(5,2)	opcional
justificativa_ia	text	opcional
modelo_ia_utilizado	varchar(80)	opcional

Coluna	Tipo	Observação
origem_calculo	varchar(30)	obrigatório — IA generativa ou contingência
data_geracao	timestamp	obrigatório

Fonte: elaborado pelos autores (2026).

Além das colunas listadas, o esquema inclui a tabela de coleção `imovel_comodidades`, que armazena as comodidades de cada imóvel, e cinco índices: `idx_imoveis_usuario`, `idx_imoveis_ativo`, `idx_reservas_imovel`, `idx_reservas_periodo` (usado pela verificação de conflito de datas) e `idx_sugestoes_imovel_data`. A unicidade do endereço de e-mail do usuário é garantida por restrição de unicidade na própria coluna, não por índice nomeado.

7 TELAS

Os protótipos de tela do SmartRent B2B já foram elaborados, mas ainda não haviam sido disponibilizados para incorporação a este relatório até o fechamento desta versão. O quadro a seguir relaciona as telas previstas para o MVP; será substituído pelas capturas reais assim que estas forem fornecidas.

Quadro 11 – Telas previstas para o MVP (pendente de anexo dos protótipos)

Tela	Propósito
Painel executivo	Visão consolidada de ocupação, receita prevista e evolução de preços sugeridos (RF10).
Gestão de imóveis	Listagem, cadastro e edição de imóveis (RF02 a RF04).
Gestão de reservas	Listagem, cadastro e cancelamento de reservas, com destaque para conflitos de data (RF05 a RF07, RF12).
Precificação inteligente	Solicitação e exibição da sugestão de preço, incluindo a justificativa da IA (RF08, RF09).

Fonte: elaborado pelos autores (2026).

REFERÊNCIAS

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 14724**: informação e documentação – trabalhos acadêmicos – apresentação. Rio de Janeiro: ABNT, 2011.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6024**: informação e documentação – numeração progressiva das seções de um documento escrito – apresentação. Rio de Janeiro: ABNT, 2012.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 10520**: informação e documentação – citações em documentos – apresentação. Rio de Janeiro: ABNT, 2023.

ASSOCIAÇÃO BRASILEIRA DE NORMAS TÉCNICAS. **NBR 6023**: informação e documentação – referências – elaboração. Rio de Janeiro: ABNT, 2018.

ORGANIZAÇÃO DAS NAÇÕES UNIDAS. **Objetivo de Desenvolvimento Sustentável 8**: trabalho decente e crescimento econômico. Nova York: ONU, 2015. Disponível em: <https://brasil.un.org/pt-br/sdgs/8>. Acesso em: 10 set. 2026.

SPRING. **Spring Boot Reference Documentation**. [S. l.]: Broadcom, [2024]. Disponível em: <https://docs.spring.io/spring-boot/>. Acesso em: 10 set. 2026.

HIBERNATE. **Hibernate ORM User Guide**. [S. l.]: Red Hat, [2024]. Disponível em: <https://hibernate.org/orm/documentation/>. Acesso em: 10 set. 2026.

GROQ. **Groq API Documentation**. [S. l.]: Groq, [2024]. Disponível em: <https://console.groq.com/docs>. Acesso em: 10 set. 2026.